

基于多目标灰狼优化算法的种植优化模型

摘要

随着农作物市场需求的不断变化，优化农作物经营策略对于乡村经济的可持续发展显得尤为关键。为帮助某乡村制定出最优种植方案，本文依据题目所给的信息，利用数学方法，建立数学模型分析不同情况下农作物的最优种植方案与销售策略，为该农村获取更多生产效益。

对于问题一，首先，本文运用 Excel 与 Python 对于各类农作物种植面积、种植成本、亩产量和销售价格等统计量进行处理。通过进行数据整合，整理出 2023 年**农作物实际种植面积与预期销售量**并对其进行数据可视化处理，分析农作物销售量分布规律；接着，运用筛选后的数据，以**最大化总利润、最小化种植分散性**为目标函数，以地块耕作条件、种植面积、轮作要求、重茬限制等为约束条件建立**多目标优化模型**；然后，引入**多目标灰狼优化算法**可估计出在超过部分滞销的情境下，该乡村 2024-2030 年农作物总利润为：**32127552.75 元**；在超过部分打折销售的情境下，该乡村 2024-2030 年农作物总利润为：**52475377.75 元**；最后，基于该模型求解出该乡村农作物最优种植方案。

对于问题二，首先，本文引入**蒙特卡洛模型**来帮助处理不确定性和区间性的数据，通过对样本的定义，包括小麦、玉米和其他作物的预期销售量增长，亩产量受气候影响的波动，以及种植成本和不同类别农作物销售价格的变化趋势，估算不同情境下农作物的预期收益、成本及风险；接着，运用蒙特卡洛模型在**灰狼优化算法的基础上**对生成解进行**模拟**，以求得**最高适应度的解**作为本问最优方案即最优种植方案。通过计算求解可估计在本问的约束条件下该乡村 2024-2030 年农作物总利润并得到该农村农作物最优种植方案。

对于问题三，首先，本文综合考量了不同农作物之间的**可替代性和互补性关系**，列举出多种类农作物的可替代性与互补性并对其进行深入分析；接着，本文对农作物的预期销售量、销售价格与种植成本之间的相互影响进行分析，运用 **Pearson 系数检验**的方法可知销售量与销售单价和种植成本间存在负线性相关，而销售量与种植成本则显示出正线性相关；最后，引入模拟数据，得出了最优的种植策略，并与问题二中所得到的数据结果进行了**对比分析**认为问题三所得到的最优种植策略会优于问题二的最优种植方案，其考虑了更多的实际因素，模型的**鲁棒性**相对更强。

综上所述，本文依据题目中所给的条件较全面地分析了多种相关因素对于农作物种植的影响，并给出了在多目标优化的条件下的农作物最优种植方案，可实现利润最大化。经过分析探究，本文的模型具有合理性，对于农作物种植事业的发展具有一定的现实意义。

关键字： 多目标优化模型 灰狼优化算法 蒙特卡洛模型 Pearson 相关系数

一、问题重述

1.1 问题背景

随着社会科技快速发展和现代信息化水平的提升，我国农业领域也在经历着深刻的变革。在这样的大背景下，我国充分发展有机农作物种植产业，不仅有利于简化田间管理流程，还有利于推进乡村经济的可持续发展。但由于农作物面临耕地资源有限、作物连续重茬减产与市场需求变化等问题，加之农作物品种众多且其亩产量与购入成本各异，因此可通过科学合理的数学方法对于农作物生产效益进行研究分析，制定动态合理的农作物最优种植策略方案，并综合考虑多个相关因素对于销售量的影响，进而实现经济效益和社会效益的双重提升。

1.2 问题要求

附件 1、2 中所给出的是我国乡村现有耕地和农作物的基本情况与 2023 年乡村的农作物种植和相关的统计数据。为使农民在今后的农作物种植中能够合理利用现有的耕地资源，减少种植风险，获得更高的收益，现需结合实际情况与附件 1、2 中所给的信息建立数学模型，解决以下问题：

问题 1：依据地块及大棚的种植规律与农作物生长规律并结合附件 1、2 给出的相关统计数据，分别分析滞销与折扣销售两种市场情境，给出 2024-2030 年的农作物最优种植方案。

问题 2：结合题目所给的经验对不同类型的农作物的预期销售量、亩产量、种植成本和销售价格的趋势预测与农作物潜在的种植风险如气候风险，制定出该乡村 2024-2030 年农作物的最优种植方案。

问题 3：依照实际考虑不同农作物之间的替代性与互补性关系，分析农作物预期销售量与销售价格和农作物种植成本的相互关系。综合考虑相关因素后制定出农作物在 2024-2030 年的最优种植方案，在模型中引入模拟数据求解并与题 2 所得到的数据进行对比分析。

二、问题分析

2.1 问题一分析

本问基于农作物滞销与折扣销售的两种市场情境下，对 2024-2030 年农作物的最优种植方案进行分析探究。首先对数据进行预处理，整合了不同地块类型和农作物种类的销售量数据。其次，通过评估不同地块类型的预期销售量与市场需求情况，建立了一个

多目标农作物种植结构优化模型，以最大化利润、最小化作物分散性，并确保种植面积不低于设定阈值为优化目标，并考虑了多个约束条件。最后利用多目标灰狼优化算法(MOGWO) 求解得到能在在两种市场情景下实现 2024-2030 年总利润最大化的最优种植方案。

2.2 问题二分析

本问题针对农作物的预期销售量、亩产量、种植成本和销售价格的不确定性及潜在的种植风险进行分析，为制定 2024-2030 年农作物的最优种植方案。首先，本文需要对相关的随机变量数据进行定义模拟，包括小麦、玉米和其他作物的预期销售量增长，亩产量受气候影响的波动情况，以及种植成本和不同类别农作物销售价格的变化趋势。接着，本文运用蒙特卡洛模拟方法生成大量随机样本，估算在不同情境下的预期收益、成本及风险。然后，结合灰狼优化算法，利用蒙特卡洛模拟的结果通过迭代搜索问题的最优解。通过不断更新算法输出具有最高适应度的解作为农作物最优种植方案。

2.3 问题三分析

本问在问题二的基础上进一步考虑了不同农作物之间的可替代性、互补性以及预期销售量、价格和种植成本之间的相关性，从而制定出更优的农作物种植策略，并与前一问题的策略进行了比较分析。首先，本文分析具有可替代性和互补性的农作物，考虑多种农作物之间的互补性特征。通过应用 Pearson 相关系数和进行相关系数 R 检验，揭示了影响因素之间的相关性以帮助得出最优农作物种植策略。

文章总体思路如下图所示：

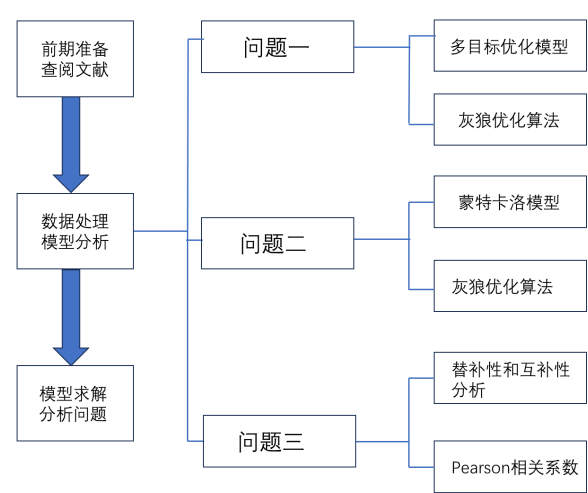


图 1 文章总体思路图

三、模型假设

为简化问题，本文做出以下假设：

- 假设农作物的历史产量能够代表未来趋势，本文利用 2023 年的农作物总产量用于代表 2024-2030 年每年的预期销售量。
- 假设附件 1、2 提供的有关农作物种植数据如农作物是准确可靠的，可用于制定种植方案。
- 假设不同地块上的农作物种植单价为该区间内的均值。
- 假设在 2024 年至 2030 年的预测期间，没有发生任何极端天气事件、病虫害、市场崩溃、政策变动或其他不可预见的灾难性事件以导致农作物的产量显著下降。

四、符号说明

符号	说明
C	农作物总数
S	农作物预售量
P	农作物销售价格
s	季节总数
L	地块总数
T	年份总数
t	第 t 年
G	预期年增长率
$X_{c,l,t,s}$	第 t 年的第 s 季，地块 l 上种植了作物 c
$Z_{l,t}$	第 t 年水浇地是否双季种植 0-1 变量
$A_{i,j,k}$	第 i 块地在第 j 年第 k 季的种植面积
$Y_{i,j,k}$	第 i 块地在第 j 年第 k 季的单位面积产量 (亩产量)
$Q_{i,j}$	作物 j 在第 i 块地的预期销售量
$P_{i,j}$	作物 j 在第 i 块地的销售价格
$C_{i,j}$	作物 j 在第 i 块地的种植成本
$A_{\min}$	最小种植面积阈值
$A_{i,j,k}$	第 i 块地在第 j 年第 k 季的种植面积
$C_{X,i,j,k}$	第 j 年第 k 季在第 i 块地种植农作物 X 的 成本
$Y_{X,j,i,k}$	第 j 年第 k 季在第 i 块地的农作物 X 的亩 产量
$M_{X,j,k}$	第 j 年第 k 季农作物 X 的亩产量变化因子
$C_{X,i,j-1,k}$	上一年 $j-1$ 第 k 季在第 i 块地种植农作物 X 的成本

五、数据侧写

由于附件 1、2 中所给到的数据结构较为复杂，为更好地进行分析研究，本文对所提供的数据进行了数据处理。根据题目具体要求，结合附件中所给的数据和信息，本文利用 Excel 和 Python 工具从多角度对于农作物相关数据进行整合，为后续算法应用与模型建立提供便利。

5.1 不同地块类型的预期销售量情况

了解不同地块类型的预期销售量，可以帮助农民更合理地分配有限的耕地资源，有效识别不同农作物的种植风险，这为制定种植策略提供了重要依据，有利于提高农作物经济效益。不同地块类型的面积和和各预期销售量情况如图 1 所示：

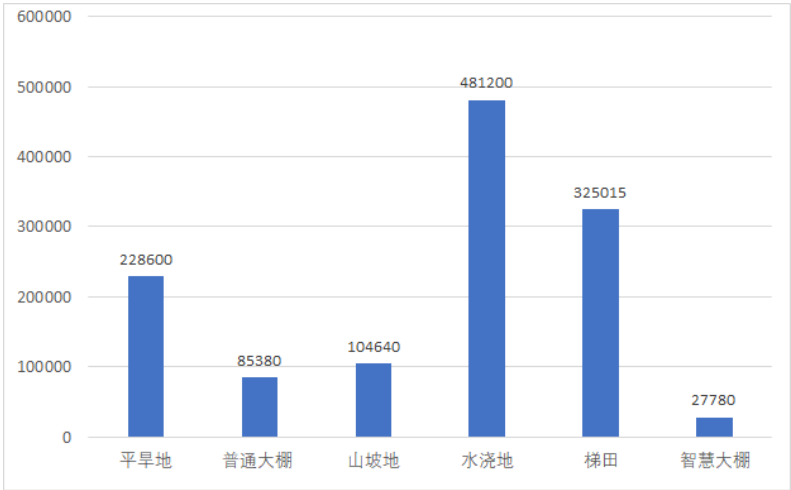


图 2 不同地块（包含大棚）的预期销售量

由图 1 可知，不同类型的地块在智慧大棚销售中的表现各不相同。其中，水浇地和梯田占据了主导地位，其含水资源都较高。相比之下，智慧大棚则相对较少，这也与智慧大棚数量较少相关。

5.2 不同种类农作物的销售情况

了解不同种类农作物的销售情况可用于帮助可以帮助农民分析各种类农作物的市场需求，优化其资源配置，调整农作物种植结构种植结构。不同种类农作物的销售情况如图 2 所示：

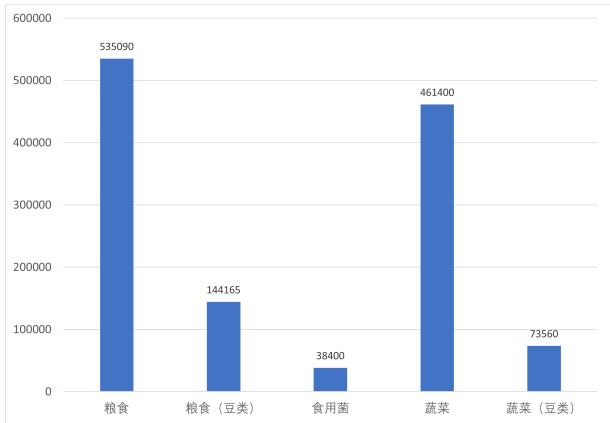


图 3 不同种类农作物的销量

从不同种类农作物的销售情况来看，粮食（包含豆类）农作物是市场上的主要需求对象，达到了 **535090 斤**，占据了大部分市场份额。而食用菌和蔬菜虽然也有市场需求，但相对较小。

六、问题一的模型的建立和求解

为依据题 1 所给的要求假定各类农作物在 2024-2030 年的预期销售量、种植成本、亩产量和销售价格相对于 2023 年保持，本文对 2023 年的农作物种植的相关统计数据整合，发现附件 2 中 2023 年农作物的实际种植情况与 2023 年的统计数据有出入，经过分析发现有多种农作物并不是在可种植地块类型上均有种植。由于本文假设了农作物历史产量可用于代表未来趋势即 2024-2030 年的预期销售量，本文运用 Excel 对两个表格中的数据进行筛选整合，统计出 2023 年的农作物总产量为 **1252615 斤**，这将作为 2024-2030 年每年的预期销售量。

6.1 多目标农作物种植结构优化模型

6.1.1 确定决策变量

由题意可知本问中的决策变量应为某块地在某一年某季节是否种植某种作物 c ，故引入 0-1 整数变量，其定义如下：

$$X_{c,l,t,s} = \begin{cases} 1, & \text{表示在第 } t \text{ 年的第 } s \text{ 季，地块 } l \text{ 上种植了作物 } c \\ 0, & \text{表示在第 } t \text{ 年的第 } s \text{ 季，地块 } l \text{ 上没有种植了作物 } c \end{cases}$$

6.1.2 确定决策变量矩阵

本文构建了一个 X 的三维矩阵为整个优化模型中的决策变量矩阵 [1]，用于表示每块地、每年、每季农作物的种植情况。现本文引入季节的概念，令 X 成为一个三维矩阵，其每个元素 $X[i, \text{year}, \text{season}]$ 表示第 i 块地，在第 year 年的第 season 季种植的作物编号。并以此多目标优化的核心部分，用于决定每年、每季每块地的种植方案。

X 是一个三维矩阵，其中：

- i 表示地块编号（第 i 块地）
- year 表示年份（如 2024 年到 2030 年）
- season 表示种植季节（如第一季和第二季）

本文将 $X[i, \text{year}, \text{season}]$ 定义为一个字典，表示该地块在某一季节种植的某个农作物及种植面积。

初始时，X 可以随机生成，也可以通过基于某些初始条件设定。在后续算法中，X 中的值会不断更新，以寻求最优解。

6.1.3 确定优化目标

1. 利润最大化

通过最大化种植收益，以确保每年的农作物总利润达到最优。其定义如下：

$$f_{\text{总利润}} = \sum_{i=1}^L \sum_{j=1}^T \sum_{k=1}^S (\min(A_{i,j,k} \times Y_{i,j,k}, Q_{i,j}) \times P_{i,j} + \max(0, A_{i,j,k} \times Y_{i,j,k} - Q_{i,j}) \times 0.5 \times P_{i,j} - A_{i,j,k} \times C_{i,j}) \quad (1)$$

- 针对第一小问中滞销部分完全浪费，不产生任何收益的利润公式为：

$$\text{利润} = \sum_{i=1}^L \sum_{j=1}^T \sum_{k=1}^S (\min(A_{i,j,k} \times Y_{i,j,k}, Q_{i,j}) \times P_{i,j} - A_{i,j,k} \times C_{i,j})$$

- 针对第二小问中所设定的滞销部分可以以 50% 的折扣价出售，本文对滞销部分进行单独处理，其利润公式为：

$$\begin{aligned} \text{利润} = & \sum_{i=1}^L \sum_{j=1}^T \sum_{k=1}^S (\min(A_{i,j,k} \times Y_{i,j,k}, Q_{i,j}) \times P_{i,j} \\ & + (\max(0, A_{i,j,k} \times Y_{i,j,k} - Q_{i,j}) \times 0.5 \times P_{i,j}) \\ & - (A_{i,j,k} \times C_{i,j})) \end{aligned}$$

其中， $X_{c,l,t,s}$ 为作物 c 在地块 l 和季节 s 的种植面积， $c_{\text{产量}}$ 和 $c_{\text{价格}}$ 为该农作物 c 的市场收益， $c_{\text{成本}}$ 为每亩的种植成本。

2. 分散性最小化

减少同一作物被分散到过多地块上，以便于农民对农作物管理。其定义如下：

$$f_{\text{分散性}} = \sum_{c=1}^C \sum_{s=1}^s (\sum_{l=1}^L \mathbf{1}(x_{c,l,t,s} > 0)) \quad (2)$$

通过限制种植作物所利用的地块数量，以确保作物不被过度分散，从而提高对作物的管理效率。

6.1.4 确定约束条件

1. 重茬限制

同一地块不能连续两年或连续两个季节种植相同的作物，即如果该作物第 t 年种了，下一年（第 t+1 年）就不能再种：

$$X_{c,l,t,s} + X_{c,l,t+1,s} \leq 1 \quad (3)$$

2. 轮作要求

在连续三年中，至少一个季节种植豆类作物：

$$\sum_{t=t_0}^{t_0+2} \sum_s X_{\text{豆类},l,t,s} \geq 1 \quad (4)$$

3. 种植面积限制

需要确保作物的种植面积不低于最小阈值，防止作物生产规模过小。

本文对于**最小种植面积阈值**进行了设定：

(a) 平旱地、梯田、山坡地的最小种植面积阈值：**4 亩**

- 这些地块适合种植粮食类作物（如小麦、玉米等），并且需要大面积才能实现机械化生产和经济效益。因此设定面积较大的地块为机械化作业留有足够的面积。

(b) 水浇地最小种植面积阈值：**2 亩**

- 水浇地可用于种植蔬菜和水稻，蔬菜和水稻的种植面积通常不需要像粮食作物那样大面积，但仍然需要一定的规模来确保产量和效益。因此设置 2 亩作为最小种植面积阈值既有助于满足产量需求，又有助于节约水资源。

(c) 大棚（普通大棚和智慧大棚）最小种植面积阈值：**0.2 亩**

- 大棚的总面积较小，且适合种植高效经济作物（如蔬菜、食用菌等）。因此较小的种植面积为合理的选择，既可高效利用，又能允许多种作物同时种植。

4. 对于平旱地、梯田和山坡地，每年只能种一季粮食类作物，且不能种植双季作物。

$$\sum_{c \in \text{粮食作物}} X_{c,l,t,1} \leq 1 \quad (5)$$

5. 对于水浇地可以种植一季水稻，或者两季蔬菜，若两季都种蔬菜，则需约束大白菜、白萝卜与红萝卜这三类蔬菜只能在第二季种植，其等式如下：

$$X_{\text{水稻},l,t,1} + Z_{l,t} \leq 1 \quad (6)$$

$$X_{\text{其他蔬菜},l,t,1} \leq Z_{l,t} \quad (7)$$

$$X_{\text{大白菜},l,t,2} + X_{\text{白萝卜},l,t,2} + X_{\text{红萝卜},l,t,2} \leq Z_{l,t} \quad (8)$$

其中， $Z_{l,t}$ 为**二元变量**，表示**是否选择双季种植蔬菜**。

6. 对于普通大棚每年可以种植作物两季。第一季可以种植蔬菜（不包括大白菜、白萝卜、红萝卜），第二季只能种植食用菌：

$$\sum_{c \in \text{蔬菜 (大白菜, 白萝卜, 红萝卜)}} X_{c,l,t,1} \leq 1 \quad (9)$$

$$X_{\text{食用菌},l,t,2} = 1 \quad (10)$$

7. 对于智慧大棚每年可以种植两季蔬菜，但大白菜、白萝卜、红萝卜均不能种植在智慧大棚中：

$$\sum_{c \in \text{蔬菜 (大白菜, 白萝卜, 红萝卜)}} X_{c,l,t,1} \leq 1 \quad (11)$$

$$\sum_{c \in \text{蔬菜 (大白菜, 白萝卜, 红萝卜)}} X_{c,l,t,2} \leq 1 \quad (12)$$

8. 预售量不能多与亩产量：

$$Y_{\text{预售}} \leq Y_{i,j,k} \quad (13)$$

综上，多目标农作物种植结构优化模型为：

$$f_{\text{总利润}} = \sum_{i=1}^L \sum_{j=1}^T \sum_{k=1}^S (\min(A_{i,j,k} \times Y_{i,j,k}, Q_{i,j}) \times P_{i,j} + \max(0, A_{i,j,k} \times Y_{i,j,k} - Q_{i,j}) \times 0.5 \times P_{i,j} - A_{i,j,k} \times C_{i,j})$$

针对第一小问：

$$\text{利润} = \sum_{i=1}^L \sum_{j=1}^T \sum_{k=1}^S (\min(A_{i,j,k} \times Y_{i,j,k}, Q_{i,j}) \times P_{i,j} - A_{i,j,k} \times C_{i,j})$$

针对第二小问：

$$\begin{aligned} \text{利润} = & \sum_{i=1}^L \sum_{j=1}^T \sum_{k=1}^S (\min(A_{i,j,k} \times Y_{i,j,k}, Q_{i,j}) \times P_{i,j} \\ & + (\max(0, A_{i,j,k} \times Y_{i,j,k} - Q_{i,j}) \times 0.5 \times P_{i,j}) \\ & - (A_{i,j,k} \times C_{i,j})) \end{aligned}$$

$$f_{\text{分散性}} = \sum_{c=1}^C \sum_{s=1}^S (\sum_{l=1}^L \mathbf{1}(x_{c,l,t,s} > 0))$$

$$s.t. \begin{cases} X_{c,l,t,s} + X_{c,l,t+1,s} \leq 1 \\ \sum_{t=t_0}^{t_0+2} \sum_s X_{豆类,l,t,s} \geq 1 \\ \sum_{c \in \text{粮食作物}} X_{c,l,t,1} \leq 1 \\ X_{水稻,l,t,1} + Z_{l,t} \leq 1 \\ X_{其他蔬菜,l,t,1} \leq Z_{l,t} \\ X_{大白菜,l,t,2} + X_{白萝卜,l,t,2} + X_{红萝卜,l,t,2} \leq Z_{l,t} \\ \sum_{c \in \text{蔬菜 (大白菜, 白萝卜, 红萝卜)}} X_{c,l,t,1} \leq 1 \\ X_{食用菌,l,t,2} = 1 \\ \sum_{c \in \text{蔬菜 (大白菜, 白萝卜, 红萝卜)}} X_{c,l,t,1} \leq 1 \\ \sum_{c \in \text{蔬菜 (大白菜, 白萝卜, 红萝卜)}} X_{c,l,t,2} \leq 1 \\ Y_{预售} \leq Y_{i,j,k} \end{cases}$$

6.2 多目标灰狼优化算法 (MOGWO)

6.2.1 算法背景

多目标灰狼优化算法 (MOGWO)[2], 是一种基于灰狼优化算法的扩展, 用于解决具有多个目标函数的优化问题。在自然界, 灰狼通过协同合作围捕猎物, 这种算法模仿了狼群的社会等级和狩猎行为, 将其应用于解空间的探索与优化。算法通过动态调整灰狼的位置, 在多个目标函数之间找到平衡。

6.2.2 多目标灰狼优化算法 (MOGWO) 步骤与流程 [3]

1. 初始化种群

初始时, 生成包含多个灰狼个体的种群, 每个灰狼个体代表一个具体的农作物种植方案。每个方案涵盖了各类农作物在不同地块和季节的种植分配。每个个体的适应度根据多目标函数进行评估, 如利润、分散性和种植面积限制。

2. 追踪猎物 (更新当前最优解)

每个灰狼根据种群中的 α 狼、 β 狼和 δ 狼的当前最优解来更新自己的位置。

其位置更新公式如下:

$$\vec{X}(t+1) = \frac{\vec{X}_\alpha(t) + \vec{X}_\beta(t) + \vec{X}_\delta(t)}{3} - A \cdot D$$

动态调整系数 A 更新公式如下:

$$A = 2a \cdot r - a$$

这个系数决定了灰狼的探索范围，确保在初期有较大的搜索空间，后期逐渐收敛到最优解。

距离向量 $\vec{D}$ 用于反应不同解之间的差异，其公式如下：

$$D = |C \cdot \vec{X}_{\text{目标}(t)} - \vec{X}_t|$$

其中， C 作为一个随机向量，用来保持解的多样性，防止过早收敛到局部最优解。

3. 包围猎物

通过不断更新解的位置，灰狼逐渐靠拢 α 狼、 β 狼和 δ 狼的最优解。随着迭代次数增加，解会逐渐收敛到 Pareto 前沿，达到多目标优化的折中点。（多个折中解）

Pareto 前沿：

$$f(x) = \{f_1(x), f_2(x), \dots, f_k(x)\}$$

其中，每个 $f_i(x)$ 代表一个目标函数，如利润、分散性和种植面积限制。

4. 停止条件与输出最优解

停止条件通常采用混合策略，包括最大迭代次数和收敛性条件：

- 最大迭代次数：通常设定 $t_{\max} = 1000$ ，当算法达到最大迭代次数时，算法停止。
- 收敛性条件：若在连续 5 次迭代中，解的变化幅度小于某个收敛阈值 $\epsilon = 10^{-5}$ ，那么则提前停止迭代。

该算法流程图与迭代图 [4] 如下：

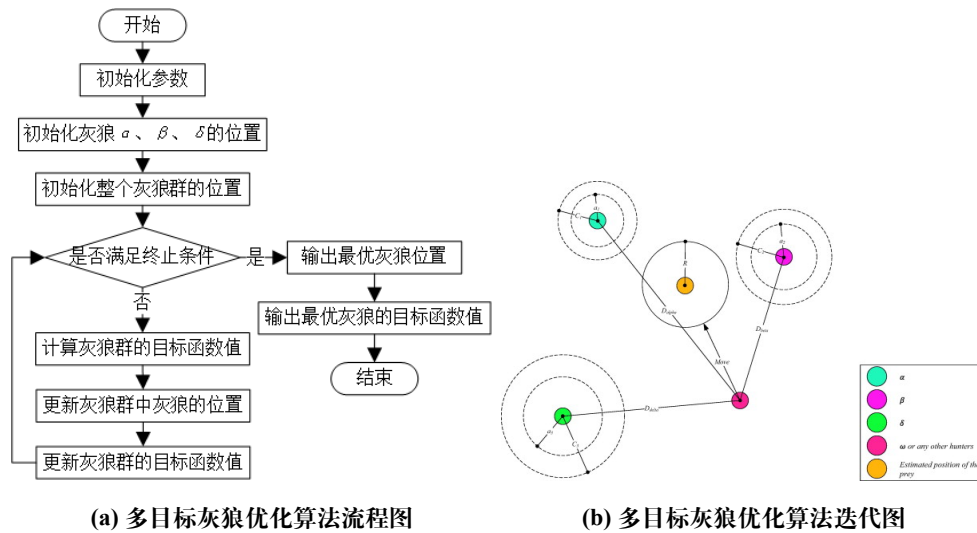


图 4 多目标灰狼优化算法

6.3 多目标灰狼优化算法（MOGWO）求解

通过 python 利用多目标灰狼优化算法进行求解，可得到两种情境下该乡村的最优种植方案：

1. 超过部分全部滞销

- 最佳总利润：**32127552.75 元**
- 该情境下部分农作物最优种植方案（由于篇幅限制，表中只展示了几类农作物的最优种植方案，其余农作物见附件 *result1_1.xlsx*）
结果如下表所示：

表 1 超出部分全部滞销的情境下 2024 年的部分农作物最优种植方案

季度	地块	农作物	种植亩数
第一季	A1	高粱	42
第一季	A2	黍子	55
第一季	A4	绿豆	20
第一季	A4	藜麦	31
第一季	B9	小麦	15
第一季	B13	玉米	26
...	...	...	...

2. 超过部分按 50% 打折出售

- 最佳总利润：**52475377.75 元**
- 该情境下部分农作物最优种植方案（由于篇幅限制，表中只展示了几类农作物的最优种植方案，其余农作物见附件 *result1_2.xlsx*）
结果如下表所示：

表 2 打折出售的情境下 2024 年的部分农作物最优种植方案

季度	地块	农作物	种植亩数
第一季	A1	黄豆	19
第一季	A1	高粱	42
第一季	A2	黍子	55
第一季	A5	黑豆	65
第一季	B1	南瓜	28
...	...	...	...

七、问题二的模型的建立和求解

本问需要在综合考虑各种农作物的预期销售量、亩产量、种植成本和销售价格的不确定性以及潜在的种植风险的情况下，制定出该乡村 2024-2030 年农作物的最优种植方案。

7.1 蒙特卡洛模型

蒙特卡洛模型作为一种基于随机抽样和模拟的方法 [5]，对于存在不确定性和区间性的数据，蒙特卡洛模型可以通过生成大量的随机样本来覆盖整个数据区间，并通过模拟这些样本的行为来估计系统的整体性能，具有一定的鲁棒性和准确性。

7.2 蒙特卡洛模型样本定义

1. 小麦和玉米未来的预期销售量有增长的趋势，平均年增长率介于 5%-10% 之间。其定义如下：

$$S_{\text{小麦},t} = S_{\text{小麦},t-1} \times (1 + G_{\text{小麦},t}) \quad (14)$$

$$S_{\text{玉米},t} = S_{\text{玉米},t-1} \times (1 + G_{\text{玉米},t}) \quad (15)$$

其中， $G_{\text{小麦},t} \in [0.05, 0.10]$ ， $G_{\text{玉米},t} \in [0.05, 0.10]$

2. 其他农作物未来每年的预期销售量相对于 2023 年大约有 $\pm 5\%$ 的变化：

$$S_{\text{其他作物},t} = S_{\text{其他作物},2023} \times (1 + G_{\text{其他作物},t}) \quad (16)$$

其中， $t \in [-0.05, 0.05]$

3. 农作物的亩产量往往会受气候等因素的影响，每年会有 $\pm 10\%$ 的变化：

$$Y_{X,j,i,k} = Y_{X,j-1,i,k} \times (1 + M_{X,j,k}) \quad (17)$$

其中， $M_{X,j,k} \in [-0.10, 0.10]$

4. 因受市场条件影响，农作物的种植成本平均每年增长 5% 左右：

$$C_{\text{总},X,i,j,k} = A_{i,j,k} \times C_{X,i,j,k} \quad (18)$$

其中 $C_{X,i,j,k}$ 可以进一步表示为： $C_{X,i,j,k} = C_{X,i,j-1,k} \times (1 + 0.05)$

5. 粮食类作物的销售价格基本稳定，假设其年增长率 $G_{\square\square,t}$ 接近于 0：

$$P_{\text{粮食},t} = P_{\text{粮食},t-1} \quad (19)$$

6. 蔬菜类作物的销售价格有增长的趋势，平均每年增长在 5% 左右：

$$P_{\text{蔬菜},t} = P_{\text{蔬菜},t-1} \times (1 + G_{\text{蔬菜},t}) \quad (20)$$

其中， $G_{\text{蔬菜},t} \approx 0.05$

7. 食用菌的销售价格稳中有降，大约每年可下降 1%-5%:

$$P_{\text{食用菌},t} = P_{\text{食用菌},t-1} \times (1 + G_{\text{食用菌},t}) \quad (21)$$

其中, $G_{\text{食用菌},t} \in [-0.01, -0.05]$

8. 羊肚菌的销售价格每年下降幅度为 5%:

$$P_{\text{羊肚菌},t} = P_{\text{羊肚菌},t-1} \times (1 + G_{\text{羊肚菌},t}) \quad (22)$$

其中, $G_{\text{羊肚菌},t} = -0.05$

7.3 与灰狼优化算法结合的求解过程

在运用灰狼优化算法求解过程中，运用蒙特卡洛模拟的数据结果，对于每一个灰狼（解）进行模拟，通过生成大量的随机样本来代表题目中所提及的不确定性因素，计算在当前解下的性能指标。

根据蒙特卡洛模拟的结果，更新灰狼算法中的 α, β, δ 等位置，然后继续迭代搜索过程。

当达到最大迭代次数或小于种植阈值时，停止算法，输出具有最高适应度的解作为问题的最优方案。

7.4 求解结果

结合蒙特卡洛模型的约束条件，本文通过 python 并运用多目标灰狼优化算法进行求解，可得到综合农作物的不确定性及其种植风险的多种因素后该乡村的最优种植方案，其结果见附件 *result2.xlsx*

八、问题三的模型的建立和求解

问题三要求在问题二的基础上，进一步考虑不同作物之间的可替代性、互补性，以及销售量、价格和种植成本之间的相关性。在综合分析相关因素后，将得出的最优种植策略与上一问进行比较分析。

8.1 替代性及互补性分析

本文对附件中具有可替代性与互补性的农作物进行具体分析。

- 具有可替代性的农作物
 - 豆类：黄豆、黑豆、豇豆、刀豆、芸豆、红豆、绿豆、爬豆属于豆类，提供类似的营养成分，可替代彼此。

- 谷物类：小麦、玉米、莠麦、大麦、水稻、谷子、高粱、黍子、荞麦作为粮食或饲料可相互替代。
- 蔬菜类：西红柿、茄子、菠菜、青椒、菜花、包菜、油麦菜、小青菜、黄瓜、生菜、辣椒、空心菜、黄心菜、芹菜、大白菜作为蔬菜均含有膳食纤维，可互相替代。
- 根茎类蔬菜：红薯、土豆、白萝卜、红萝卜可以在一定程度上替代彼此。
- 食用菌：榆黄菇、白灵菇、羊肚菌、香菇可在烹饪选择中互相替代，提供不同的风味。
- 具有互补性的农作物：
 - 氮固定作物与其他作物：黄豆、黑豆等豆类作物能够固定大气中的氮，与玉米、小麦等非固氮作物进行轮作，可用于提高土壤肥力。
 - 根茎类与叶菜类：这两类作物种植过程中所利用的土壤层次不同，例如土豆和菠菜共同种植过程中，土豆利用下层土壤，而菠菜利用表层土壤。
 - 高矮作物搭配：如高粱与菜花共同进行种植，可在物理空间上形成互补，有助于提高单位面积的产量。
 - 蔬菜与食用菌：西红柿、黄瓜等蔬菜与食用菌共同种植，可以形成一个多样化的生态系统，有助于减少病虫害危害。

8.2 相关性分析

8.2.1 Pearson 相关系数应用

Pearson 相关系数是一种衡量两个变量之间的线性关系的统计指标。其取值范围在-1 到 1 之间，其中相关系数为正值表示正相关，即一个变量的增加与另一个变量的增加呈正比例关系；相关系数为负值表示负相关，即一个变量的增加与另一个变量的减少呈正比例关系；接近于 0 的值表示两个变量之间几乎没有线性关系。其计算公式如下：

$$r = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}} \quad (23)$$

1. Pearson 相关系数求解结果其系数求解结果如下表：

表 3 Pearson 相关系数

	预期销售量与销售 单价	预期销售量与种植 成本	销售单价与种植成 本
相关系数	-0.1747	-0.3000	0.5245

由表中可知，预期销售量与销售单价和预期销售量与种植成本的相关系数均小于 0，

说明预期销售量与销售单价和预期销售量与种植成本呈负线性相关关系。相反，预期销售量与种植成本的相关系数大于 0，说明其呈现正线性相关关系。

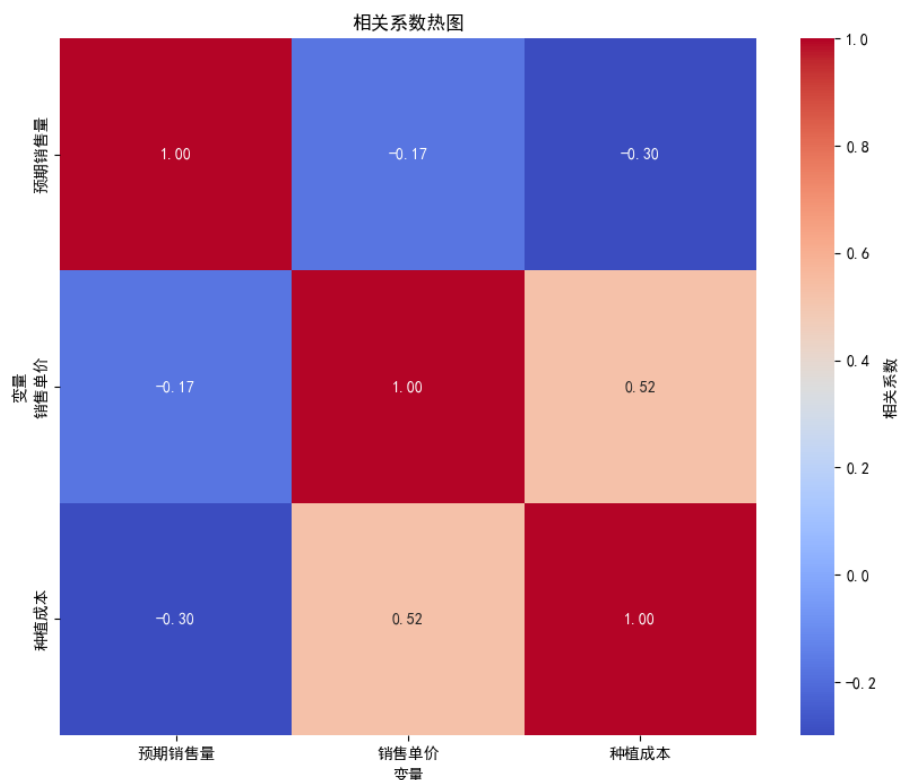


图 5 Pearson 相关系数热图

2. Pearson 相关系数检验本文对所得到的结果 Pearson 相关系数检验，提出原假设 Pearson 相关系 $R \neq 0$ ，并设定置信水平为 90%

本文利用 Python 进行 Pearson 相关系数检验，其检验结果如下表所示：

表 4 Pearson 相关系数检验

	预期销售量与销售 单价	预期销售量与种植 成本	销售单价与种植成 本
P 值	0.2063	0.000***	0.000***

由表中可得，预期销售量与种植成本和销售单价与种植成本的 P 值约为 0，远小于置信水平，可判断预期销售量与种植成本和销售单价与种植成本均呈现显著的线性相关关系。相反，预期销售量与销售单价的 P 值大于之置信水平 0.05，不可拒绝原假设，可说明预期销售量与销售单价之间的线性相关水平较弱。

8.3 模型的比较与分析

综合多种因素条件进行对比分析，本文认为题目 3 所求得的结果相较于问题 2 更加准确，思考方面如下：

- 结果的动态性和适应性分析
 - 问题二是在给定的不确定性条件下通过算法寻求最优解，其更偏向于静态的最优解。
 - 问题三需要考虑农作物间的相互作用与市场条件的动态变化，其更具动态性和适应性。
- 应用情景
 - 问题二基于当前的市场和环境条件下，提供的农作物种植策略更适用于短期至中期的规划。
 - 问题三综合考虑了作物间的长期相互作用关系和市场条件的变化因素，其种植策略更适合长期规划。
- 鲁棒性

问题三考虑了更多的实际因素，其模型的鲁棒性更强，更能适应实际农业生产中多样变化的条件。

九、模型的评价

9.1 模型的优点

- 多目标优化模型与灰狼优化算法具有较好的全局搜索能力和收敛速度，本文为制定最优种植方案需要综合考虑多个因素如预期销售量、亩产量、种植成本和销售价格等，运用该模型与算法能够有效地求解复杂的多目标优化问题。
- 题目中为多种农作物给出了多个不确定性的条件，蒙特卡洛模型能有效处理不确定性和复杂性问题，从而可有效帮助评估不同种植方案的风险和收益，具有鲁棒性。
- Pearson 相关系数，可以用于度量连续变量之间的线性关系，以判断两个变量之间的相关性，结果易于理解解释。

9.2 模型的缺点

- 2024-2030 年农作物的预期销售量可能会随着实际市场需求上升或下降，本文假设其均 2023 年的产量一致，可能会对生成结果产生误差。
- 多目标灰狼优化算法对参数设置和初始种群的选择较为敏感，对于如第二问与

动态相关的优化问题，该模型难以直接应用。

- 蒙特卡洛模型需要较长时间进行模拟，对输入数据的质量和准确性要求高，可能导致结果不准确。

参考文献

- [1] 贺莉, 刘庆怀. 多目标优化理论与连续化方法[M]. 北京: 科学出版社, 2015.
- [2] 张志东. 一种改进灰狼优化算法研究及应用[M]. 北京: 北京理工大学出版社, 2020.
- [3] WU L, TIAN J, LIU Y, et al. Multi-objective planting structure optimisation in an irrigation area using a grey wolf optimisation algorithm[M]. China: MDPI, 2024.
- [4] MIRJALILI S, MIRJALILI S M, LEWIS A. Grey wolf optimizer[M]. Australia: MDPI, 2011.
- [5] 克里斯托弗·Z·穆尼. 蒙特卡罗模拟[M]. 上海: 上海人民出版社, 2018.
- [6] 司守奎, 孙玺菁. 数学建模算法与应用[M]. 北京: 国防工业出版社, 2011.

附录 A 文件列表

文件名	功能描述
源程序.py	本论文的程序代码
<i>result1_1.xlsx</i>	问题一第一小问求得的最优种植方案
<i>result1_2.xlsx</i>	问题一第二小问求得的最优种植方案
<i>result2.xlsx</i>	问题二求得的最优种植方案

附录 B 代码

源程序.py

```
1 # -*- coding: utf-8 -*-
2 """Untitled2.ipynb
3
4 Automatically generated by Colab.
5
6 Original file is located at
7     https://colab.research.google.com/drive/109
8     Ar5aosW8RzQDxinOypqDL9xuZcMIk0
9 """
10 from google.colab import files
11 uploaded = files.upload()
12
13 from google.colab import files
14 uploaded = files.upload()
15
16 from google.colab import files
17 uploaded = files.upload()
18
19 from google.colab import files
20 uploaded = files.upload()
21
22 from google.colab import files
23 uploaded = files.upload()
```

```

24
25 # Commented out IPython magic to ensure Python compatibility.
26 # %reset -f
27
28 import pandas as pd
29 import numpy as np
30 import random
31 import matplotlib.pyplot as plt
32
33
34 # 读取文件
35 land_info = pd.read_excel('地块信息.xlsx')
36 crop_info = pd.read_excel('整理后的农作物信息.xlsx')
37 stats_2023 = pd.read_excel('2023统计数据.xlsx')
38 expected_sales_data = pd.read_excel('总和2.xlsx')
39 planting_2023 = pd.read_excel('2023种植情况(1).xlsx')
40
41 # 提取需要的列
42 land_info_clean = land_info[['地块名称', '地块类型', '地块面积
    /亩']]
43 crop_info_clean = crop_info[['作物编号', '作物名称', '作物类型
    ', '种植耕地']]
44 stats_2023_clean = stats_2023[['作物编号', '作物名称', '亩产量
    /斤', '种植成本/(元/亩)', '销售单价/(元/斤)', '地块类型']]
45 expected_sales_data = expected_sales_data[['作物名称', '总和'
    ]]
46 planting_2023_clean = planting_2023[['种植地块', '作物编号', '
    作物名称', '种植面积/亩', '种植季次']]
47
48 # 生成地块编号与地块名称的映射
49 land_names = land_info_clean['地块名称'].tolist() # 从地块信
    息中提取地块名称列表
50 land_id_map = {idx: name for idx, name in enumerate(land_names
    )} # 编号从0开始
51 print("地块名称映射:", land_id_map)

```

```

52
53 # 创建作物名称和编号的映射
54 crop_name_to_id_map = {name: crop_id for crop_id, name in zip(
    crop_info_clean['作物编号'], crop_info_clean['作物名称'])}
55
56 # 打印作物名称到编号的映射，确保映射正确
57 print(crop_name_to_id_map)
58
59 # 从 land_info_clean 中提取地块名称和地块类型
60 land_types = land_info_clean['地块类型'].tolist()
61
62 # 生成地块编号与地块类型的映射
63 land_id_type_map = {idx: land_type for idx, land_type in
    enumerate(land_types)}
64
65 # 打印地块编号到地块类型的映射，确保映射正确
66 print("地块编号到地块类型的映射:", land_id_type_map)
67
68 # 使用 .loc 显式地更新 DataFrame 的指定列
69 stats_2023_clean.loc[:, '作物编号'] = stats_2023_clean['作物名
    称'].map(crop_name_to_id_map)
70
71 # 删除作物名称列
72 stats_2023_clean = stats_2023_clean.drop(columns=['作物名称'])
73
74 # 删除包含 NaN 的行
75 planting_2023_clean = planting_2023_clean.dropna(subset=['种植
    地块'])
76
77 # 定义作物编号集合
78 water_rice = {16} # 水稻
79 vegetable_set = set(range(17, 35)) # 普通蔬菜
80 root_veg = {35, 36, 37} # 大白菜、白萝卜、红萝卜
81
82 # 粮食作物（除了水稻）的编号集合（1到15）

```

```

83 cereal_set = set(range(1, 16)) # 包含编号1到15的作物
84
85 legume_set = {1, 2, 3, 4, 5, 17, 18, 19}
86
87 mushroom_set = {38, 39, 40, 41} # 食用菌
88
89 def no_repeated_cropping(X, planting_2023_clean, land_names):
90     # 遍历每个地块
91     for i, land_name in enumerate(land_names): # 直接使用地块
92         # 名称
93         if land_name is None:
94             continue # 跳过没有匹配的地块
95
96         # 从2023年的种植记录中找到当前地块的记录
97         matching_rows = planting_2023_clean.loc[
98             planting_2023_clean['种植地块'] == land_name
99         ]
100
101         # 第一步：检查同一块地在同一年内的两个季节是否重茬
102         for year in range(len(X[i])): # 遍历每一年
103             # 获取该地块当前年份两个季节的作物编号
104             season_1_crops = {crop[0] for crop in X[i][year
105 ][0]} # 第一季作物编号集合
106             season_2_crops = {crop[0] for crop in X[i][year
107 ][1]} # 第二季作物编号集合
108
109             # 如果第一季和第二季有相同作物编号，则违反重茬限制
110             if season_1_crops & season_2_crops:
111                 return False # 同一年两个季节种相同作物，违反
112                 限制
113
114         # 第二步：检查同一块地在不同年份同一季节是否重茬
115         for year in range(1, len(X[i])): # 从第二年开始
116             for season in range(len(X[i][year])): # 遍历每个
117                 季节

```

```

113         # 获取当前年份和前一年该季节的作物编号
114         current_crops = {crop[0] for crop in X[i][year
][season]} # 当前年份该季节的作物编号集合
115         previous_crops = {crop[0] for crop in X[i][
year-1][season]} # 前一年同季节的作物编号集合
116
117         # 如果同一季节的作物编号有重叠，则违反跨年份的
重茬限制
118         if current_crops & previous_crops:
119             return False # 跨年份的同季节种植了相同作
物，违反重茬限制
120
121         # 第三步：特殊处理单季作物
122         for idx, row in matching_rows.iterrows():
123             if row['种植季次'] == '单季':
124                 # 假设单季作物在第一季种植，跳过第二季的检查
125                 continue
126
127         return True # 如果没有发现重茬现象，返回 True
128
129 def legume_rotation(X, planting_2023_clean, legume_set,
land_info_clean):
130     for i in range(len(X)): # 遍历每个地块
131         planting_history = []
132
133         # 处理2023年的作物记录，从 DataFrame 获取地块名称
134         plot_name = land_info_clean['地块名称'].iloc[i]
135         matching_rows = planting_2023_clean[
planting_2023_clean['种植地块'] == plot_name]
136
137         if matching_rows.empty:
138             print(f"Warning: No planting data for plot {
plot_name} in 2023.")
139         else:
140             for _, row in matching_rows.iterrows():

```

```

141         planting_history.append(row['作物编号'])
142
143     # 添加未来7年的种植记录
144     for year in range(len(X[i])):
145         for season in range(len(X[i][year])):
146             crops = X[i][year][season].keys() # 获取每季
的作物编号
147             planting_history.extend(crops)
148
149     # 检查每个三年周期
150     for start_year in range(len(planting_history) - 6): #
每3年6个季节
151         three_year_period = planting_history[start_year:
start_year + 6] # 获取三年内的作物
152         if not any(crop in legume_set for crop in
three_year_period):
153             return False # 如果三年内没有种植豆类作物，返
回 False
154
155     return True # 如果所有地块都满足轮作要求，返回 True
156
157 def single_season_cereal_restriction(X, land_info_clean,
cereal_set):
158     for i in range(len(X)): # 遍历每个地块
159         try:
160             land_type = land_info_clean['地块类型'].iloc[i] #
获取当前地块的类型
161         except KeyError:
162             print(f"Error: '地块类型' not found for plot {i}."
)
163             continue
164
165     # 仅在乎旱地、梯田、山坡地等地块上施加限制
166     if land_type in ['平旱地', '梯田', '山坡地']:
167         for year in range(len(X[i])): # 遍历每年

```

```

168         season_1_crop = X[i][year][0] # 第一季作物
169         season_2_crop = X[i][year][1] # 第二季作物
170
171         # 检查第一季是否种植了非粮食作物
172         for crop in season_1_crop.keys():
173             if crop not in cereal_set:
174                 return False # 如果第一季种植了非粮食
作物，违反限制
175
176         # 检查第二季是否种植了非粮食作物
177         for crop in season_2_crop.keys():
178             if crop not in cereal_set:
179                 return False # 如果第二季种植了非粮食
作物，违反限制
180
181         # 确保一年只能种一季粮食作物
182         season_1_in_cereal = any(crop in cereal_set
for crop in season_1_crop.keys())
183         season_2_in_cereal = any(crop in cereal_set
for crop in season_2_crop.keys())
184         if season_1_in_cereal and season_2_in_cereal:
185             return False # 违反了一年只能种一季粮食作
物的限制
186
187         return True # 如果没有违反限制，返回 True
188
189 def irrigation_land_restriction(X, land_info_clean, water_rice
, vegetable_set, root_veg):
190     for i in range(len(X)): # 遍历每个地块
191         try:
192             land_type = land_info_clean['地块类型'].iloc[i] #
获取当前地块的类型
193         except KeyError:
194             print(f"Error: '地块类型' not found for plot {i}."
)

```

```

195         continue
196
197     # 仅对水浇地施加限制
198     if land_type == '水浇地':
199         for year in range(len(X[i])): # 遍历每一年
200             season_1_crop = X[i][year][0] # 第一季作物 (
字典形式)
201             season_2_crop = X[i][year][1] # 第二季作物 (
字典形式)
202
203             # 检查方案1: 单季水稻, 第二季必须为空
204             if any(crop in water_rice for crop in
season_1_crop.keys()): # 第一季种植水稻
205                 if season_2_crop: # 如果第二季不是空, 则
违反限制
206                     return False
207
208             # 检查方案2: 双季蔬菜, 第一季为普通蔬菜, 第二
季为根类蔬菜
209             elif any(crop in vegetable_set for crop in
season_1_crop.keys()): # 第一季种植普通蔬菜
210                 if not any(crop in root_veg for crop in
season_2_crop.keys()): # 第二季必须是根类蔬菜
211                     return False
212
213             # 如果第一季既不是水稻也不是蔬菜, 返回False
214             else:
215                 return False # 如果第一季不是水稻或蔬菜,
违反限制
216
217     return True # 满足条件时返回True
218
219 def normal_greenhouse_restriction(X, land_info_clean,
vegetable_set, mushroom_set):
220     for i in range(len(X)): # 使用 len(X) 来获取地块数量

```

```

221         land_type = land_info_clean['地块类型'].iloc[i] # 获
    取当前地块的类型
222         if land_type == '普通大棚': # 检查是否为普通大棚
223             for year in range(len(X[i])): # 遍历每年
224                 # 获取该年份两个季节的作物编号
225                 season_1_crops = {crop[0] for crop in X[i][
    year][0]} # 第一季作物编号集合
226                 season_2_crops = {crop[0] for crop in X[i][
    year][1]} # 第二季作物编号集合
227
228                 # 第一季必须是蔬菜
229                 if not any(crop in vegetable_set for crop in
    season_1_crops):
230                     return False # 如果第一季不是蔬菜, 违反限
    制
231
232                 # 第二季必须是食用菌
233                 if not any(crop in mushroom_set for crop in
    season_2_crops):
234                     return False # 如果第二季不是食用菌, 违反
    限制
235             return True
236
237 def smart_greenhouse_restriction(X, land_info_clean,
    vegetable_set, root_veg):
238     for i in range(len(X)): # 遍历每块地
239         land_type = land_info_clean['地块类型'].iloc[i] # 获
    取当前地块的类型
240         if land_type == '智慧大棚': # 确保是智慧大棚
241             for year in range(len(X[i])): # 遍历每年
242                 # 获取该年份两个季节的作物编号
243                 season_1_crops = {crop[0] for crop in X[i][
    year][0]} # 第一季作物编号集合
244                 season_2_crops = {crop[0] for crop in X[i][
    year][1]} # 第二季作物编号集合

```

```

245
246         # 第一季和第二季都必须是蔬菜
247         if not all(crop in vegetable_set for crop in
season_1_crops) or \
248             not all(crop in vegetable_set for crop in
season_2_crops):
249             return False # 如果任一季不是蔬菜，违反限
制
250
251         # 检查是否种植了大白菜、白萝卜、红萝卜
252         if any(crop in root_veg for crop in
season_1_crops) or \
253             any(crop in root_veg for crop in
season_2_crops):
254             return False # 不能种植大白菜、白萝卜、红
萝卜，违反限制
255         return True
256
257 def calculate_profit(X, stats_2023_clean, land_info_clean,
discount=False):
258     total_profit = 0 # 初始化总利润
259
260     # 遍历每个地块
261     for i in range(len(X)):
262         land_type = land_info_clean['地块类型'].iloc[i] # 获
取当前地块的类型
263
264         # 遍历每一年和每个季节
265         for year in range(len(X[i])):
266             for season in range(len(X[i][year])):
267                 for crop_area in X[i][year][season]: # 遍历每
个作物及其种植面积
268                     if isinstance(crop_area, tuple) and len(
crop_area) == 2:
269                         crop, area = crop_area

```

```

270
271         # 确保作物和面积的有效性
272         if not isinstance(crop, (int, float)) or
not isinstance(area, (int, float)):
273             print(f"Error: Invalid crop or area at
plot {i}, year {year}, season {season}")
274             continue
275
276         # 确保地块类型选择正确
277         land_type_to_use = "智慧大棚" if (
land_type == "智慧大棚" and season == 1) else land_type
278
279         # 获取该作物的亩产量、成本、销售单价
280         crop_stats = stats_2023_clean[
281             (stats_2023_clean['作物编号'] == crop)
&
282             (stats_2023_clean['地块类型'] ==
land_type_to_use)
283         ]
284
285         if crop_stats.empty:
286             print(f"Warning: No stats found for
crop {crop} on land type {land_type_to_use}")
287             continue
288
289         # 提取亩产量、种植成本、销售单价
290         try:
291             yield_per_acre = float(crop_stats['亩
产量/斤'].values[0])
292             planting_cost = parse_range(crop_stats
['种植成本/(元/亩)'].values[0])
293             price_per_pound = parse_range(
crop_stats['销售单价/(元/斤)'].values[0])
294         except (IndexError, ValueError):
295             print(f"Error processing stats for

```

```

crop {crop} on land type {land_type_to_use}")
296         continue
297
298         # 计算收入和成本
299         revenue = area * yield_per_acre *
price_per_pound
300         cost = area * planting_cost
301
302         # 如果有滞销折扣
303         if discount:
304             revenue *= 0.5
305
306         # 计算利润并加入总利润
307         profit = revenue - cost
308         total_profit += profit
309
310     return total_profit
311
312 # 解析种植成本或销售单价范围的辅助函数
313 def parse_range(value):
314     if isinstance(value, str) and '-' in value:
315         min_val, max_val = map(float, value.split('-'))
316         return (min_val + max_val) / 2 # 返回中间值
317     return float(value) # 如果不是范围，则直接返回浮点值
318
319 from collections import defaultdict
320
321 def calculate_dispersion(X):
322     total_dispersion = 0
323
324     # 遍历每一年
325     for year in range(len(X[0])): # 假设所有地块都有相同的年份数量
326         for season in range(len(X[0][year])): # 假设所有地块
都有相同的季节数量

```

```

327         crop_land = defaultdict(set) # 记录每个作物出现在的地块
328
329         # 遍历每块地
330         for i in range(len(X)):
331             if season in range(len(X[i][year])): # 检查当前季节是否存在
332                 for crop, area in X[i][year][season]: # 遍历每个作物编号
333                     crop_land[crop].add(i) # 记录作物出现在的地块编号
334
335             # 计算分散度（即每个作物种植的地块数量）
336             for lands in crop_land.values():
337                 total_dispersion += len(lands) # 计算总的分散度
338
339         return total_dispersion
340
341 def check_area_constraints(X, land_info_clean):
342     for i in range(len(X)): # 遍历每块地
343         land_type = land_info_clean['地块类型'].iloc[i]
344
345         for year in range(len(X[i])): # 遍历每一年
346             for season in range(len(X[i][year])): # 遍历每个季节
347                 crops_for_season = X[i][year][season] # 获取当季作物及面积的列表
348
349                 for crop, area in crops_for_season:
350                     if land_type in ['平旱地', '梯田', '山坡地'] and area < 3:
351                         return False # 平旱地、梯田、山坡地上一类作物面积不得小于3亩
352

```

```

353         elif land_type == '水浇地' and area < 2:
354             return False # 水浇地上一类作物面积不
得小于2亩
355
356         elif land_type in ['普通大棚', '智慧大棚']
and area < 0.2:
357             return False # 大棚中的每类作物面积不
得小于0.2亩
358
359         return True # 如果所有条件都满足, 返回 True
360
361 import numpy as np
362
363 def initialize_solution_with_constraints(X_template,
land_info_clean, planting_grouped, legume_set, cereal_set,
water_rice, vegetable_set, root_veg, mushroom_set):
364     for i in range(len(X_template)): # 遍历每个地块
365         land_type = land_info_clean['地块类型'].iloc[i] # 获
取地块类型
366
367         for year in range(len(X_template[i])): # 遍历每年
368             for season in range(len(X_template[i][year])): #
遍历每个季节
369                 crops_for_season = [] # 当前季节作物的列表
370
371                 # 确定种多少种作物 (1-3种)
372                 num_crops = np.random.randint(1, 4)
373
374                 for _ in range(num_crops): # 根据地块类型和季
节选择作物
375                     if land_type in ['平旱地', '梯田', '山坡地
']:
376                         crop_id = np.random.choice(list(
cereal_set)) # 选择粮食作物
377                         crop_area = np.random.randint(20, 81)

```

```

# 面积范围
378         elif land_type == '水浇地':
379             if season == 0:
380                 crop_id = np.random.choice([16] +
list(vegetable_set)) # 水稻或蔬菜
381                 crop_area = np.random.randint(20,
81) if crop_id == 16 else np.random.randint(1, 21)
382             else:
383                 crop_id = np.random.choice(list(
root_veg)) # 根类蔬菜
384                 crop_area = np.random.randint(1,
21)
385         elif land_type == '普通大棚':
386             crop_id = np.random.choice(list(
vegetable_set)) if season == 0 else np.random.choice(list(
mushroom_set))
387             crop_area = np.round(np.random.uniform
(0.2, 0.6), 1) # 大棚作物
388         elif land_type == '智慧大棚':
389             crop_id = np.random.choice(list(
vegetable_set)) # 智慧大棚的两季都为蔬菜
390             crop_area = np.round(np.random.uniform
(0.2, 20), 1)
391
392         crops_for_season.append((crop_id,
crop_area)) # 存储作物编号和面积
393
394         X_template[i][year][season] = crops_for_season
# 将作物信息存储到 X_template
395
396     return X_template
397
398 def initialize_wolf_population_from_2023(num_wolves,
X_template, land_info_clean, planting_grouped, legume_set,
cereal_set, water_rice, vegetable_set, root_veg,

```

```

mushroom_set):
399     population = [] # 初始化狼群
400     for idx in range(num_wolves):
401         # 每生成一个狼就打印一次
402         print(f"生成第 {idx + 1} 个狼")
403         solution = initialize_solution_with_constraints(
404             X_template, land_info_clean, planting_grouped,
legume_set, cereal_set, water_rice, vegetable_set, root_veg,
mushroom_set
405         )
406         population.append(solution) # 将生成的方案加入狼群
407
408     return population
409
410 def update_wolf_position(wolf, alpha_wolf, iteration, max_iter
, land_info_clean, legume_set, cereal_set, water_rice,
vegetable_set, root_veg, mushroom_set):
411     print(f"正在更新第 {iteration + 1}/{max_iter} 代狼的位置")
412     new_wolf = wolf.copy()
413     a = 2 - 2 * (iteration / max_iter) # 线性递减参数
414
415     for i in range(len(new_wolf)):
416         for year in range(len(new_wolf[i])):
417             for season in range(len(new_wolf[i][year])):
418                 r1, r2 = np.random.random(), np.random.random
()
419
420                 A = 2 * a * r1 - a
421                 C = 2 * r2
422
423                 alpha_crop_id, alpha_area = alpha_wolf[i][year
][season] # Alpha狼的作物编号和面积
424                 wolf_crop_id, wolf_area = new_wolf[i][year][
season] # 当前狼的作物编号和面积
425
426                 if isinstance(alpha_crop_id, (int, float)) and

```

```

426         isinstance(wolf_crop_id, (int, float)):
427             D_alpha = abs(C * alpha_crop_id -
wolf_crop_id)
428             new_crop_id = max(1, min(alpha_crop_id - A
* D_alpha, 41)) # 保证作物编号在范围内
429             new_wolf[i][year][season] = (new_crop_id,
wolf_area)
430
431             # 进行约束检查
432             valid_solution = False
433             attempts = 0
434             while not valid_solution and attempts < 20: #
最多尝试20次以满足约束
435                 if (no_repeated_cropping(new_wolf,
planting_2023_clean, land_info_clean) and
436                     legume_rotation(new_wolf,
planting_2023_clean, legume_set, land_info_clean) and
437                     single_season_cereal_restriction(
new_wolf, land_info_clean, cereal_set) and
438                     irrigation_land_restriction(new_wolf,
land_info_clean, water_rice, vegetable_set, root_veg) and
439                     normal_greenhouse_restriction(new_wolf
, land_info_clean, vegetable_set, mushroom_set) and
440                     smart_greenhouse_restriction(new_wolf,
land_info_clean, vegetable_set, root_veg)):
441                         valid_solution = True # 如果解满足所
有约束，设置为有效解
442                         else:
443                             # 如果不满足约束，重新生成作物
444                             new_wolf[i][year][season] =
regenerate_crop(land_info_clean, i, season, vegetable_set,
root_veg, cereal_set, mushroom_set)
445                             attempts += 1
446
447             return new_wolf

```

```

447
448 def regenerate_crop(land_info_clean, i, season, vegetable_set,
    root_veg, cereal_set, mushroom_set):
449     land_type = land_info_clean['地块类型'].iloc[i]
450     if land_type in ['平旱地', '梯田', '山坡地']:
451         return (np.random.choice(list(cereal_set)), np.random.
    randint(20, 80)) # 粮食作物
452     elif land_type == '水浇地':
453         crop_type = np.random.choice(['water_rice', 'vegetable
    '])
454         if crop_type == 'water_rice':
455             return (16, np.random.randint(20, 80)) # 水稻
456         else:
457             return (np.random.choice(list(vegetable_set)), np.
    random.randint(2, 20)) # 蔬菜
458     elif land_type == '普通大棚':
459         if season == 0:
460             return (np.random.choice(list(vegetable_set)), np.
    round(np.random.uniform(0.2, 20), 1)) # 蔬菜
461         else:
462             return (np.random.choice(list(mushroom_set)), np.
    round(np.random.uniform(0.2, 0.6), 1)) # 食用菌
463     elif land_type == '智慧大棚':
464         return (np.random.choice(list(vegetable_set)), np.
    round(np.random.uniform(0.2, 20), 1)) # 两季蔬菜
465
466 def mogwo_optimization(num_wolves, max_iter, X_template,
    land_info_clean, planting_grouped, legume_set, cereal_set,
    water_rice, vegetable_set, root_veg, mushroom_set, epsilon=1
    e-5):
467     # 初始化狼群
468     population = initialize_wolf_population_from_2023(
469         num_wolves,
470         X_template,
471         land_info_clean,

```

```

472     planting_grouped,
473     legume_set,
474     cereal_set,
475     water_rice,
476     vegetable_set,
477     root_veg,
478     mushroom_set
479 )
480
481 alpha_wolf = None # 当前最优解
482 alpha_profit = None # 当前最优利润, 初始化为None
483 prev_profit = None # 记录上一次的最优利润, 初始化为None
484
485 for iter in range(max_iter):
486     for wolf in population:
487         # 计算当前解的利润
488         profit = calculate_profit(wolf, stats_2023_clean,
land_info_clean)
489
490         # 更新Alpha狼 (最优解)
491         if alpha_profit is None or profit > alpha_profit:
492             alpha_wolf = wolf
493             alpha_profit = profit
494
495         # 如果是第一次计算, 初始化 prev_profit
496         if prev_profit is None:
497             prev_profit = alpha_profit
498
499         # 检查利润变化
500         profit_diff = abs(alpha_profit - prev_profit)
501
502         if profit_diff < epsilon:
503             print(f"Convergence achieved at iteration {iter
+1}, Best Profit = {alpha_profit}")
504             break # 提前停止

```

```

505
506     prev_profit = alpha_profit # 更新前一轮的最优利润
507
508     # 更新狼群位置
509     for i in range(len(population)):
510         population[i] = update_wolf_position(
511             population[i], alpha_wolf, iter, max_iter,
512             land_info_clean, legume_set, cereal_set,
water_rice,
513             vegetable_set, root_veg, mushroom_set)
514
515     # 输出当前迭代信息
516     print(f"Iteration {iter+1}/{max_iter}, Best Profit = {
alpha_profit}")
517
518     # 最终输出
519     print(f"\nFinal Best Solution with Profit = {alpha_profit}
")
520     return alpha_wolf, alpha_profit
521
522 def initialize_X_template(num_plots, num_years, num_seasons):
523     """
524     初始化一个四维数组 X_template
525     该数组用于存储每个地块、每一年、每个季节的作物编号和种植面
积。
526     """
527     # 创建一个空的四维数组（地块编号、年份、季节、作物编号），
初始化为空列表
528     X_template = [[[[[] for _ in range(num_seasons)] for _ in
range(num_years)] for _ in range(num_plots)]
529     return X_template
530
531 num_plots = 54 # 54块地
532 num_years = 7 # 7年
533 num_seasons = 2 # 每年2个季节

```

```

534
535 # 初始化 X_template
536 X_template = initialize_X_template(num_plots, num_years,
    num_seasons)
537
538 # 调用 MOGWO 优化算法
539 best_solution, best_profit = mogwo_optimization(
540     num_wolves=30, # 狼群的数量
541     max_iter=50, # 最大迭代次数
542     X_template=X_template, # 初始方案模板
543     land_info_clean=land_info_clean, # 地块信息
544     planting_grouped=planting_2023_clean.groupby('种植地块'),
    # 2023年种植数据
545     legume_set=legume_set, # 豆类作物集合
546     cereal_set=cereal_set, # 粮食作物集合
547     water_rice=water_rice, # 水稻作物
548     vegetable_set=vegetable_set, # 蔬菜作物集合
549     root_veg=root_veg, # 根类蔬菜
550     mushroom_set=mushroom_set, # 蘑菇作物集合
551     epsilon=1e-5 # 收敛条件
552 )
553
554 # 输出最优解
555 print("最佳方案: ", best_solution)
556 print("最佳利润: ", best_profit)
557
558 def get_midpoint_from_range(range_value):
559     # 如果是字符串且包含 '-', 说明是区间, 解析区间
560     if isinstance(range_value, str) and '-' in range_value:
561         try:
562             lower, upper = map(float, range_value.split('-'))
563             return (lower + upper) / 2
564         except ValueError:
565             print(f"无法解析区间: {range_value}")
566             return 0

```

```

567     # 如果不是字符串，直接返回该数值
568     elif isinstance(range_value, (int, float)):
569         return range_value
570     else:
571         print(f"未知类型的数据: {range_value}")
572         return 0
573
574 import numpy as np
575
576
577
578 def initialize_solution_with_constraints(X_template,
    land_info_clean, legume_set, cereal_set, water_rice,
    vegetable_set, root_veg, mushroom_set):
579     for i in range(len(X_template)): # 遍历每个地块
580         land_type = land_info_clean['地块类型'].iloc[i] # 获
    取地块类型
581         land_area = land_info_clean['地块面积/亩'].iloc[i] #
    获取该地块的最大面积
582
583         for year in range(len(X_template[i])): # 遍历每一年
584             previous_crops = set() # 用来存储过去一年及当年的
    作物，防止重茬
585
586             # 将前一年的作物加入到 previous_crops 中，防止重茬
587             if year > 0:
588                 for last_season in X_template[i][year - 1]:
589                     for crop in last_season:
590                         previous_crops.add(crop[0]) # 只存储
    作物编号
591
592             for season in range(len(X_template[i][year])): #
    遍历每个季节
593                 crops_for_season = [] # 当前季节作物的列表
594                 remaining_area = land_area # 每季节的剩余种植

```

面积初始化为地块面积

```
595         season_crops = set() # 防止同一季节重复种植同  
种作物
```

```
596  
597         # 确定种多少种作物（最多3种）  
598         num_crops = np.random.randint(1, 4) # 随机选  
择1-3种作物
```

```
599         num_crops = min(num_crops, 3) # 强制限制作物  
种类数量不超过 3
```

```
600  
601         for _ in range(num_crops):  
602             crop_id = None  
603             crop_area = None  
604  
605             # 根据地块类型选择作物  
606             if land_type in ['平旱地', '梯田', '山坡地  
' ]:
```

```
607                 available_crops = list(set(cereal_set)  
- previous_crops - season_crops) # 避免重茬和重复作物  
608                 if available_crops and remaining_area  
> 0: # 检查是否有可种植的作物和足够的面积
```

```
609                     crop_id = np.random.choice(  
available_crops)
```

```
610                     if remaining_area >= 20:  
611                         crop_area = np.random.randint  
(20, min(81, remaining_area + 1)) # 分配合理的面积
```

```
612                     else:  
613                         crop_area = remaining_area #  
剩余面积不够大时，使用剩余面积
```

```
614                     elif land_type == '水浇地':  
615                         available_crops = list(set([16] + list  
(vegetable_set)) - previous_crops - season_crops)
```

```
616                         if available_crops and remaining_area  
> 0:
```

```
617                             crop_id = np.random.choice(
```

```

available_crops)
618         if remaining_area >= 20:
619             crop_area = np.random.randint
(20, min(81, remaining_area + 1))
620         else:
621             crop_area = remaining_area
622         elif land_type == '普通大棚':
623             available_crops = list(set(
vegetable_set) - previous_crops - season_crops) if season ==
0 else list(set(mushroom_set) - previous_crops -
season_crops)
624         if available_crops and remaining_area
> 0:
625             crop_id = np.random.choice(
available_crops)
626             crop_area = round(np.random.
uniform(0.3, min(0.6, remaining_area)), 1) # 精确到一位小数
627         elif land_type == '智慧大棚':
628             available_crops = list(set(
vegetable_set) - previous_crops - season_crops)
629             if available_crops and remaining_area
> 0:
630                 crop_id = np.random.choice(
available_crops)
631                 crop_area = round(np.random.
uniform(0.3, min(0.6, remaining_area)), 1) # 精确到一位小数
632
633             # 确保作物不重茬、不重复种植并有足够面积
634             if crop_id is not None and crop_area <=
remaining_area and crop_id not in previous_crops and crop_id
not in season_crops:
635                 crops_for_season.append((crop_id,
crop_area))
636                 remaining_area -= crop_area # 更新剩
余的种植面积

```

```

637         previous_crops.add(crop_id) # 防止重
        茬
638         season_crops.add(crop_id) # 防止同一
        季节重复种植同种作物
639
640         # 如果当前季节没有作物分配，确保分配至少一种作
        物
641         if not crops_for_season and remaining_area >
        0:
642             available_crops = list(set(cereal_set) -
        previous_crops)
643             if available_crops:
644                 crop_id = np.random.choice(
        available_crops) # 选择粮食作物作为备用作物
645                 crop_area = min(remaining_area, np.
        random.randint(20, 81)) # 分配一个合理的种植面积
646                 crops_for_season.append((crop_id,
        crop_area))
647                 remaining_area -= crop_area
648
649                 # 将种植方案保存到 X_template 中
650                 X_template[i][year][season] = crops_for_season
651
652         return X_template
653
654
655 num_plots = 54 # 54块地
656 num_years = 7 # 7年
657 num_seasons = 2 # 每年2个季节
658
659
660 X_template = np.empty((num_plots, num_years, num_seasons),
        dtype=object)
661
662

```

```

663
664 # 调用
665 X_solution = initialize_solution_with_constraints(
666     X_template, land_info_clean, legume_set, cereal_set,
667     water_rice, vegetable_set, root_veg, mushroom_set
668 )
669
670 def get_midpoint_from_range(range_value):
671     if isinstance(range_value, str) and '-' in range_value:
672         try:
673             lower, upper = map(float, range_value.split('-'))
674             return (lower + upper) / 2
675         except ValueError:
676             print(f"无法解析区间: {range_value}")
677             return 0
678     elif isinstance(range_value, (int, float)):
679         return range_value
680     else:
681         print(f"未知类型的数据: {range_value}")
682         return 0
683
684 def calculate_profit(wolf, stats_2023_clean,
685                     expected_sales_data, case):
686     total_profit = 0
687
688     # 遍历方案中的每个地块、每年、每个季节
689     for plot in wolf:
690         for year in plot:
691             for season in year:
692                 for crop_id, area in season:
693                     # 跳过 crop_id 为 None 的情况
694                     if crop_id is None:
695                         print("发现 crop_id 为 None, 跳过该作物")

```

```

695         continue
696
697         # 根据 crop_id 找到作物名称
698         crop_data = stats_2023_clean[
stats_2023_clean['作物编号'] == crop_id]
699
700         # 检查是否找到对应的作物数据
701         if not crop_data.empty:
702             crop_data = crop_data.iloc[0]
703             crop_name = crop_data['作物名称'] #
使用作物名称来匹配
704             yield_per_acre = crop_data['亩产量/斤'
]
705             cost_range = crop_data['种植成本/(元/
亩)']
706             price_range = crop_data['销售单价/(元/
斤)']
707
708             # 使用 get_midpoint_from_range 解析成
本和单价
709             cost = get_midpoint_from_range(
cost_range)
710             price = get_midpoint_from_range(
price_range)
711
712             # 使用作物名称来匹配
expected_sales_data 中的总和
713             expected_sales_row =
expected_sales_data[expected_sales_data['作物名称'] ==
crop_name]
714
715             # 检查预期销售量是否存在
716             if not expected_sales_row.empty:
717                 expected_sales =
expected_sales_row['总和'].values[0] # 获取预期销售量

```

```

718         else:
719             print(f"作物 {crop_name} 在
expected_sales_data 中未找到")
720             expected_sales = 0 # 如果未找到,
默认值为0
721
722             # 计算当前产量
723             total_yield = yield_per_acre * area
724
725             # 情况 1: 产量超过预期销售量, 滞销
726             if case == 1:
727                 if total_yield > expected_sales:
728                     revenue = price *
expected_sales # 只能卖掉预期销售量部分
729                 else:
730                     revenue = price * total_yield
731
# 全部卖掉
732
# 情况 2: 产量超过预期销售量, 按50%价
格出售
733
734             elif case == 2:
735                 if total_yield > expected_sales:
736                     revenue = price *
expected_sales + 0.5 * price * (total_yield - expected_sales
)
737
738                 else:
739                     revenue = price * total_yield
740
741             # 计算种植成本
742             expenses = cost * area
743
744             # 计算总利润
745             total_profit += (revenue - expenses)
746         else:
747             print(f"作物编号 {crop_id} 在
stats_2023_clean 中未找到")

```

```

745
746     return total_profit
747
748
749 def mogwo_optimization(num_wolves, max_iter, X_template,
    land_info_clean, stats_2023_clean, legume_set, cereal_set,
    water_rice, vegetable_set, root_veg, mushroom_set,
    expected_sales_data, case, epsilon=10000):
750     # 初始化狼群
751     population = [initialize_solution_with_constraints(
    X_template, land_info_clean, legume_set, cereal_set,
    water_rice, vegetable_set, root_veg, mushroom_set) for _ in
    range(num_wolves)]
752
753     alpha_wolf = None # 当前最优解
754     alpha_profit = -float('inf') # 当前最优利润，初始化为负无
    穷
755     prev_profit = None # 记录上一次的最优利润
756
757     for iter in range(max_iter):
758         # 对每一只狼的解进行扰动
759         for wolf_index in range(num_wolves):
760             wolf = population[wolf_index]
761
762             # 在当前解基础上随机扰动，生成新解
763             new_solution =
    initialize_solution_with_constraints(X_template,
    land_info_clean, legume_set, cereal_set, water_rice,
    vegetable_set, root_veg, mushroom_set)
764
765             # 计算扰动后解的利润
766             new_profit = calculate_profit(new_solution,
    stats_2023_clean, expected_sales_data, case)
767
768             # 更新狼的解，如果新解更优

```

```

769         if new_profit > alpha_profit:
770             alpha_wolf = new_solution
771             alpha_profit = new_profit
772
773         # 更新狼群中的该狼解
774         population[wolf_index] = new_solution
775
776         # 如果是第一次计算，初始化 prev_profit
777         if prev_profit is None:
778             prev_profit = alpha_profit
779
780         # 检查利润变化，判断是否终止迭代
781         profit_diff = abs(alpha_profit - prev_profit)
782
783         # 输出当前迭代信息
784         print(f"Iteration {iter+1}/{max_iter}, Best Profit = {
alpha_profit}, Profit Difference = {profit_diff}")
785
786         # 如果利润的变化小于预定的阈值，提前终止
787         if profit_diff < epsilon:
788             print(f"Convergence achieved at iteration {iter
+1}, stopping early. Best Profit = {alpha_profit}")
789             break
790
791         # 更新新一轮的最优利润
792         prev_profit = alpha_profit
793
794         # 最终输出
795         print(f"\nFinal Best Solution with Profit = {alpha_profit}
")
796         return alpha_wolf, alpha_profit
797
798
799
800 # 调用代码

```

```

801 num_plots = 54 # 地块数量
802 num_years = 7 # 年份数量
803 num_seasons = 2 # 每年2个季节
804
805 # 初始化 X_template
806 def initialize_X_template(num_plots, num_years, num_seasons):
807     return [[[[[] for _ in range(num_seasons)] for _ in range(
808         num_years)]] for _ in range(num_plots)]
809
810 X_template = initialize_X_template(num_plots, num_years,
811     num_seasons)
812
813 # 情况1：滞销利润最大化
814 case = 1
815 best_solution_case1, best_profit_case1 = mogwo_optimization(
816     num_wolves=30,
817     max_iter=50,
818     X_template=X_template,
819     land_info_clean=land_info_clean,
820     stats_2023_clean=stats_2023_clean,
821     legume_set=legume_set,
822     cereal_set=cereal_set,
823     water_rice=water_rice,
824     vegetable_set=vegetable_set,
825     root_veg=root_veg,
826     mushroom_set=mushroom_set,
827     expected_sales_data=expected_sales_data,
828     case=case
829 )
830 print("最佳方案（情况1：滞销）：", best_solution_case1)
831 print("最佳利润（情况1：滞销）：", best_profit_case1)
832
833 # 情况2：降价出售利润最大化
834 def generate_case2_from_case1(case1_solution, stats_2023_clean

```

```

, expected_sales_data):
834     case2_solution = []
835     for plot in case1_solution:
836         new_plot = []
837         for year in plot:
838             new_year = []
839             for season in year:
840                 new_season = []
841                 for crop_id, area in season:
842                     if crop_id is not None:
843                         # 对种植面积进行轻微的扰动
844                         new_area = area * np.random.uniform
(0.9, 1.1)
845                         new_season.append((crop_id, new_area))
846                     else:
847                         new_season.append((crop_id, area))
848                 new_year.append(new_season)
849             new_plot.append(new_year)
850         case2_solution.append(new_plot)
851
852     return case2_solution
853
854
855 case = 2
856 best_solution_case2, best_profit_case2 = mogwo_optimization(
857     num_wolves=30,
858     max_iter=50,
859     X_template=X_template,
860     land_info_clean=land_info_clean,
861     stats_2023_clean=stats_2023_clean,
862     legume_set=legume_set,
863     cereal_set=cereal_set,
864     water_rice=water_rice,
865     vegetable_set=vegetable_set,
866     root_veg=root_veg,

```

```

867     mushroom_set=mushroom_set,
868     expected_sales_data=expected_sales_data,
869     case=case
870 )
871 print("最佳利润（情况2：降价出售）：", best_profit_case2)
872
873 case2_solution = generate_case2_from_case1(best_solution_case1
      , stats_2023_clean, expected_sales_data)
874 best_profit_case2_generated = calculate_profit(case2_solution,
      stats_2023_clean, expected_sales_data, case=2)
875 print("最佳方案（情况2：降价出售）：", case2_solution)
876
877 import csv
878
879 def export_solution_to_csv(solution, file_name='best_solution.
      csv'):
880     # 打开 CSV 文件
881     with open(file_name, mode='w', newline='', encoding='utf-8
      ') as file:
882         writer = csv.writer(file)
883
884         # 写入表头
885         writer.writerow(["Plot Index", "Year", "Season", "Crop
      ID", "Crop Area"])
886
887         # 遍历最佳方案的每个地块、年份、季节
888         for plot_index, plot in enumerate(solution):
889             for year_index, year in enumerate(plot):
890                 for season_index, season in enumerate(year):
891                     for crop_id, crop_area in season:
892                         # 写入作物信息
893                         writer.writerow([plot_index,
      year_index, season_index, crop_id, crop_area])
894
895     print(f"方案已成功导出为 {file_name}")

```

```
896
897 # 分别导出两种方案
898 export_solution_to_csv(best_solution_case1, file_name='
      best_solution_case1.csv')
899 export_solution_to_csv(best_solution_case2, file_name='
      best_solution_case2.csv')
900
901 from google.colab import files
902
903 files.download('best_solution_case1.csv')
904 files.download('case2_solution.csv')
```